

# Object-Oriented Programming in Python: Learning by Building a Bank

If you are studying programming for the first time, the phrase "object-oriented programming" can sound intimidating. It is often shortened to **OOP**, and it is one of the most important ways of organizing code in Python and many other languages. The good news is that the core idea is genuinely intuitive once you connect it to the everyday world.

This article teaches OOP from scratch using one running example: a simple bank. We will build it up piece by piece, and by the end you will have written account types, given them behavior, protected their data, and made them work together. No prior OOP knowledge is assumed. Every code snippet here has been run and verified, so you can type them out and watch them work.

---

## Part 1: Why Object-Oriented Programming Exists

Imagine you are writing a program to manage bank accounts. Without OOP, you might keep a customer's balance in a plain variable and write loose functions to change it:

```
python

balance = 0

def deposit(amount):
    global balance
    balance += amount

deposit(100)
print(balance)    # 100
```

This works for *one* account. But a real bank has thousands of accounts, each with its own owner, balance, and history. You would end up with `balance1`, `balance2`, `balance3`, and a tangle of functions that all need to know which balance to touch. It quickly becomes a mess.

OOP offers a cleaner mental model. Instead of scattering data and functions everywhere, you **bundle related data and the actions that operate on it into a single unit**. For a bank, that unit is an "account" that knows its own owner, its own balance, and how to deposit or withdraw from itself. Each account manages its own affairs.

This bundling mirrors how we think about the real world. A bank account in real life *is* a thing that has properties (an owner, a balance) and can do things (accept deposits, allow withdrawals). OOP lets your code reflect that natural structure.

---

## Part 2: Classes and Objects — The Blueprint and the Building

Two words sit at the heart of OOP: **class** and **object**. The relationship between them is the single most important idea to grasp, so let's anchor it with an analogy.

Think of a **class** as a *blueprint* for a house. The blueprint is not a house you can live in — it is a detailed plan describing what every house built from it will have: rooms, doors, windows. From one blueprint you can build many actual houses.

An **object** is an *actual house* built from that blueprint. Each house is real, you can live in it, and each one is independent — painting one house red does not change the others, even though they all came from the same plan.

In banking terms: `BankAccount` is the class (the blueprint describing what every account has and does), and Alice's specific account is an object (also called an **instance**) built from that blueprint. Bob's account is a different object built from the very same blueprint.

Here is the simplest possible class in Python:

```
python

class BankAccount:
    pass          # 'pass' just means "empty for now"

account = BankAccount()    # build an object from the blueprint
print(type(account))       # <class '__main__.BankAccount'>
```

The keyword `class` starts the blueprint. Writing `BankAccount()` with parentheses tells Python to actually construct an object from that blueprint. Right now our account is empty and useless, so let's give it some properties.

---

## Part 3: Attributes and the `__init__` Method

A real account needs to know its owner and balance. These pieces of data that belong to an object are called **attributes**. We set them up using a special method named `__init__` (pronounced "dunder init", short for "double underscore init").

`__init__` is the **initializer**. Python runs it automatically the moment you create a new object, and its job is to set up that object's starting attributes. Think of it as the instructions the construction crew follows the instant a new house is built — "install these specific fixtures."

```
python
```

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance
```

There are two things here that almost every beginner pauses on, so let's address them directly.

### What is `self`?

`self` refers to *the specific object being worked on right now*. When Alice's account runs a method, `self` means "Alice's account." When Bob's account runs the same method, `self` means "Bob's account." It is how each object refers to itself.

Returning to the blueprint analogy: the blueprint says "paint *this house's* front door blue." The word "this house" changes meaning depending on which house the crew is currently building. `self` is exactly that "this house." Python passes it in automatically — you never supply it yourself when calling a method.

So `self.owner = owner` means "store the value of `owner` as an attribute on *this particular object*."

### What does `balance=0` do?

That is a **default value**. If someone creates an account without specifying a starting balance, it defaults to 0. This makes the parameter optional.

Now we can create accounts that actually hold information:

```
python

alice = BankAccount("Alice", 100)
bob = BankAccount("Bob")           # no balance given, so it defaults to 0

print(alice.owner, alice.balance)  # Alice 100
print(bob.owner, bob.balance)      # Bob 0
```

Notice that `alice` and `bob` are two independent objects. They came from the same class but hold completely separate data. Changing Alice's balance will never affect Bob's. This is the bundling promise of OOP delivered: each object carries its own data.

---

## Part 4: Methods — Giving Objects Behavior

Attributes are what an object *has*. **Methods** are what an object can *do*. A method is simply a function defined inside a class, and it always takes `self` as its first parameter so it can access and modify that object's own attributes.

Let's teach our account to deposit, withdraw, and report its balance:

python

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        return self.balance

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds")
            return self.balance
        self.balance -= amount
        return self.balance

    def show_balance(self):
        print(f"{self.owner}'s balance: ${self.balance}")
```

Now watch an account manage itself:

python

```
carol = BankAccount("Carol", 50)
carol.deposit(100)           # balance becomes 150
carol.withdraw(30)           # balance becomes 120
carol.show_balance()         # Carol's balance: $120
carol.withdraw(1000)         # Insufficient funds
```

Look at how natural `carol.deposit(100)` reads — almost like an English sentence: "Carol, deposit 100." Each method works on `carol`'s own balance because `self` inside the method is Carol's account. We call the methods with no `self` argument; Python supplies it behind the scenes, filling it with the object on the left of the dot.

The `withdraw` method also shows something powerful: an object can **protect its own rules**. The account itself refuses to let you withdraw more than you have. The logic that guards the balance lives right next to the balance it protects.

---

## Part 5: Validation — Letting Objects Guard Their Own Data

Because methods are the gatekeepers to an object's data, they are the perfect place to enforce rules. A deposit of a negative number should not be allowed, for instance. We can

build that rule directly into the method:

```
python

class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        if amount <= 0:
            print("Deposit must be positive")
            return
        self.balance += amount
        print(f"Deposited ${amount}. New balance: ${self.balance}")

dan = BankAccount("Dan")
dan.deposit(100)    # Deposited $100. New balance: $100
dan.deposit(-5)    # Deposit must be positive
```

The account now defends itself against invalid input. Anyone using this class gets the safety check for free, every single time, without having to remember to write it themselves. This is a big part of why OOP leads to more reliable programs.

---

## Part 6: Inheritance — Building New Types from Existing Ones

Real banks offer more than one kind of account. A savings account earns interest; a checking account might allow a small overdraft. These are all *kinds of* bank account — they share the basic behavior (deposit, withdraw, track a balance) but each adds something special.

OOP handles this elegantly through **inheritance**. Inheritance lets a new class take on all the attributes and methods of an existing class, then add or change whatever it needs. The existing class is called the **parent** (or base class), and the new one is the **child** (or subclass).

The analogy here is family resemblance. A child inherits traits from a parent automatically, but also has their own unique characteristics. You do not need to redefine the shared traits; they come for free.

Let's make `SavingsAccount` inherit from `BankAccount`:

```
python
```

```

class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds")
            return
        self.balance -= amount

class SavingsAccount(BankAccount):      # the (BankAccount) means "inherit from
    def __init__(self, owner, balance=0, interest_rate=0.02):
        super().__init__(owner, balance)    # let the parent do its setup
        self.interest_rate = interest_rate  # then add our own extra attribute

    def add_interest(self):
        interest = self.balance * self.interest_rate
        self.balance += interest
        print(f"Added ${interest:.2f} interest. Balance: ${self.balance:.2f}")

```

Two new things appear here, both worth understanding clearly.

### Writing `(BankAccount)` after the class name

`class SavingsAccount(BankAccount):` is how you declare inheritance. It says "a `SavingsAccount` is a `BankAccount` and should have everything a `BankAccount` has."

### What is `super().__init__(...)`?

`super()` refers to the parent class. So `super().__init__(owner, balance)` means "run the parent's initializer to handle the setup it already knows how to do" — in this case storing the owner and balance. This saves us from copying that code. After the parent finishes its part, the child adds its own extra attribute, the interest rate.

Now the savings account has *everything* the base account had, plus its new ability:

```

python

eve = SavingsAccount("Eve", 1000, 0.05)
eve.deposit(500)           # inherited from BankAccount — works automatically
eve.add_interest()         # Added $75.00 interest. Balance: $1575.00

```

We never wrote a `deposit` method inside `SavingsAccount`, yet `eve.deposit(500)` works perfectly. It was inherited from the parent. That is the labor-saving magic of inheritance: write shared behavior once, reuse it across many related types.

---

## Part 7: Overriding — Changing Inherited Behavior

Sometimes a child needs to do something *differently* from its parent. A checking account, for example, might permit withdrawing slightly more than the balance, up to an overdraft limit. We can replace the inherited `withdraw` method with our own version — this is called **overriding**.

```
python

class CheckingAccount(BankAccount):
    def __init__(self, owner, balance=0, overdraft_limit=100):
        super().__init__(owner, balance)
        self.overdraft_limit = overdraft_limit

    def withdraw(self, amount):          # same name as the parent's method — this
        if amount > self.balance + self.overdraft_limit:
            print("Exceeds overdraft limit")
            return
        self.balance -= amount
        print(f"Withdrew ${amount}. Balance: ${self.balance}")

frank = CheckingAccount("Frank", 100)
frank.withdraw(150)      # Withdrew $150. Balance: $-50
```

Frank withdrew \$150 from a \$100 balance, dropping to -\$50, because his checking account allows an overdraft. The parent class would have refused this, but the child's own `withdraw` method takes priority. When you define a method with the same name as one in the parent, the child's version wins. Same idea as a child forming their own opinion that differs from their parent's.

---

## Part 8: Encapsulation — Protecting Data from the Outside

So far, anyone can reach in and change an account's balance directly:

```
python

account.balance = 1_000_000    # uh oh — no rules, no checks
```

That defeats the safety we built into our methods. **Encapsulation** is the principle of hiding an object's internal data and forcing all access to go through controlled methods. It is like a

real bank: you cannot walk into the vault and rearrange the cash yourself: you must go through a teller who follows the rules.

In Python, prefixing an attribute name with two underscores marks it as **private**, signaling it should not be touched from outside the class:

```
python

class SafeAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.__balance = balance          # the double underscore makes it private

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):                 # a controlled way to read the balance
        return self.__balance

grace = SafeAccount("Grace", 100)
grace.deposit(50)
print(grace.get_balance())               # 150

print(grace.__balance)                   # AttributeError – you cannot reach it directly
```

Now the balance can only be changed through `deposit` (which validates the amount) and only read through `get_balance`. The internal data is sealed off, and every interaction with it passes through methods that enforce the rules. This keeps your objects in a valid, trustworthy state.

---

## Part 9: A Friendly Display with `__str__`

When you print an object by default, Python shows something unhelpful like `<__main__.BankAccount object at 0x7f...>`. You can define a special method called `__str__` to control what gets shown instead, giving your object a human-readable description:

```
python
```

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def __str__(self):
        return f"Account({self.owner}, ${self.balance})"

heidi = BankAccount("Heidi", 200)
print(heidi)      # Account(Heidi, $200)
```

`__str__` runs automatically whenever the object needs to be turned into text, such as when you `print` it. It is a small touch that makes your objects far pleasanter to work with and debug.

---

## Part 10: The Four Pillars, Summarized in Banking Terms

Everything we built maps onto the four classic principles of OOP. Seeing them named, now that you have used each one, ties the whole picture together:

- **Encapsulation** — bundling data with the methods that protect it, and hiding the internals. *Our account guards its balance and only lets you change it through `deposit` and `withdraw`.*
- **Inheritance** — building new classes from existing ones to reuse code. *`SavingsAccount` and `CheckingAccount` inherited the basics of `BankAccount` for free.*
- **Polymorphism** — the idea that different objects can respond to the same method name in their own way. *Calling `withdraw` on a checking account behaves differently from calling it on a basic account, even though the method has the same name.* (The word literally means "many forms.")
- **Abstraction** — exposing simple, meaningful actions while hiding the messy details. *You call `eve.add_interest()` without needing to know the formula inside.*

Do not worry about memorizing these terms. What matters is that you have now *done* each of them. The vocabulary will settle in naturally as you keep practicing.

---

## Summary

Here is the whole journey in a nutshell:

- A **class** is a blueprint; an **object** (or instance) is a real thing built from that blueprint. One `BankAccount` class can produce countless independent account objects.

- `__init__` is the initializer that sets up an object's starting **attributes** the moment it is created.
- `self` is how an object refers to itself, so each object works with its own data.
- **Methods** are functions inside a class that give objects behavior, like `deposit` and `withdraw`, and they are the natural place to enforce rules.
- **Inheritance** lets a child class reuse a parent's attributes and methods, and `super()` lets the child tap into the parent's setup.
- **Overriding** lets a child replace an inherited method to behave differently.
- **Encapsulation** (with private `__attributes`) hides internal data so it can only be changed through trustworthy methods.
- Special methods like `__str__` make objects friendlier to use.

The big takeaway is that OOP lets your code mirror the way you already think about the world: as things that have properties and can perform actions. A bank account, a student, a car, a playlist — each becomes a class, each real one becomes an object, and your program starts to read like a description of the problem you are solving. That clarity is why OOP has remained one of programming's most enduring ideas.

The best next step is to build your own small class from scratch — perhaps a `Student` with a name and grades, or a `Car` with a fuel level it tracks. Give it a few attributes, a couple of methods, and a rule or two to enforce. The concepts here only truly click once your own fingers have typed them.